# CREDIT CARD FRAUD DETECTION USING MACHINE LEARNING

MICRO PROJECT REPORT

submitted by

**Athul Krishna A S (IES24CD013)**
**Fathimasafa K A (IES24CD021)**
**Jannath K R (IES24CD027)**

to

*APJ Abdul Kalam Technological University*

*in partial fulfillment of the requirements for the award of the degree of*

*Bachelor of Technology in Computer Science and Engineering (Data Science)*




**Department of Computer Science and Engineering (Data Science)**

**IES College of Engineering, Chittilappilly**

Thrissur - 680 551

**October 2026**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)**
**IES COLLEGE OF ENGINEERING, CHITTILAPPILLY**

# CERTIFICATE

This is to certify that the report entitled "CREDIT CARD FRAUD DETECTION USING MACHINE LEARNING" submitted by Athul Krishna A S (IES24CD013), Fathimasafa K A (IES24CD021) and Jannath K R (IES24CD027) to the APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (Data Science) is a bonafide record of the work carried out under my guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

| | |
|---|---|
| **Faculty In Charge** | **Head of Department** |
| **Name: [Faculty name]** | **Name: [HOD name]** |
| **Signature: _____** | **Signature: _____** |
| **Date: _____** | **Date: _____** |

(Seal of Department of Computer Science and Engineering (Data Science))

# ABSTRACT

The rapid growth of digital payment systems and online financial transactions has led to a significant increase in credit card fraud, resulting in substantial financial losses for individuals and financial institutions. This project applies data analysis and machine learning techniques to identify potentially fraudulent credit card transactions by analyzing historical transaction data. The system examines transaction attributes such as transaction time, transaction amount and a set of anonymized features derived from the original transaction details, in order to identify unusual or suspicious patterns that may indicate fraud.

Through data preprocessing, exploratory data analysis and a Random Forest classification model built in R, the project aims to distinguish fraudulent transactions from legitimate ones. Because fraud is extremely rare in real data, special care is taken to handle class imbalance and to evaluate the model with suitable measures such as precision, recall, F1-score and ROC-AUC rather than accuracy alone. The results are presented through an interactive Shiny web application that supports dataset exploration, model performance review, single-transaction prediction and batch prediction using CSV files.

The proposed solution demonstrates how data-driven decision making can assist banks and financial organizations in improving fraud detection, minimizing financial losses and enhancing customer trust, while reducing false alarms.

Keywords: Credit Card Fraud Detection, Data Analysis, Financial Security, Fraud Prevention, Predictive Analytics, Random Forest, R, Shiny.

# TABLE OF CONTENTS

Chapter I: INTRODUCTION I

Chapter II: OBJECTIVES II

Chapter III: SYSTEM DESIGN III

Chapter IV: METHODOLOGY IV

Chapter V: SYSTEM REQUIREMENTS V

Chapter VI: RESULTS AND OUTPUTS VI

Chapter VII: CONCLUSION VII

REFERENCES REF

# Chapter I: INTRODUCTION

With the growth of online shopping and digital payments, credit cards are used for millions of transactions every day. Unfortunately, this has also increased the number of unauthorized transactions. Credit card fraud causes large financial losses to cardholders, banks and merchants, and damages customer trust.

Traditional rule-based fraud detection systems depend on fixed conditions written by humans, for example blocking transactions above a certain amount. Such systems struggle to identify new and complex fraud patterns, and they can generate many false alarms. Machine learning offers a data-driven alternative: a model learns patterns from past transactions whose outcome is already known and then uses those patterns to judge new transactions.

This project, Credit Card Fraud Detection Using Machine Learning, analyzes historical credit card transaction data and classifies each transaction as either a legitimate transaction or a potentially fraudulent transaction. The project is implemented in R using RStudio, with dplyr for data manipulation, ggplot2 for visualization, caret for model tuning and evaluation, randomForest for classification, and Shiny for the web interface.

The dataset used is the publicly available Kaggle Credit Card Fraud Detection dataset, which contains 284,807 transactions made by European cardholders over two days, of which only 492 are fraudulent (about 0.17%). Each transaction has a Time value, an Amount value, 28 anonymized numeric features (V1 to V28) obtained through Principal Component Analysis (PCA), and a Class label (0 for legitimate, 1 for fraud). The original details are anonymized to protect customer privacy.

The project covers dataset analysis, visualization, preprocessing, class imbalance handling, model training, model evaluation, feature importance and prediction through a web application. It is a demonstration on historical public data; it is not connected to any bank and does not monitor real transactions in real time.

# Chapter II: OBJECTIVES

The main objective of the project is to develop a machine learning based system that analyzes historical credit card transactions and classifies them as legitimate or potentially fraudulent, and presents the results through clear visualizations and an interactive application.

The specific objectives are:

- To analyze the transaction dataset. To examine the number of records, columns, data types, missing values, duplicate rows and the proportion of fraudulent transactions.
- To understand class imbalance. To study how rare fraud is compared with legitimate transactions and why this makes model evaluation difficult.
- To visualize transaction data. To use ggplot2 charts for class distribution, transaction amount distribution, amount comparison between classes and other exploratory views.
- To preprocess the data. To validate the data, prepare the target column, remove only unnecessary rows (exact duplicates and rows with missing values) and prevent data leakage.
- To split the data correctly. To create reproducible stratified training and testing sets so that the model is tested on unseen data.
- To handle class imbalance. To apply resampling only to the training data and never to the test data.
- To build a Random Forest model. To train and tune a Random Forest classifier and save it so it can be reused without retraining.
- To evaluate the model. To compute the confusion matrix, accuracy, precision, recall, specificity, F1-score, ROC-AUC and precision-recall AUC from real test predictions.
- To identify important features. To show which input features the model relies on most, without claiming causation.
- To predict single transactions. To let a user select a sample transaction or enter feature values and obtain a predicted class and fraud probability.
- To predict in batches. To let a user upload a CSV file, validate it, obtain predictions and download the results.
- To demonstrate data-driven fraud detection. To show how machine learning can support banks in early identification of suspicious activity while reducing false alarms.

# Chapter III: SYSTEM DESIGN

## 3.1 System Architecture

The architecture describes the flow of data from the source dataset to the final prediction. The supplied architecture screenshot is shown as Figure 1.

Note. This supplied architecture image also depicts Python/scikit-learn, Streamlit, Hadoop HDFS and Spark/PySpark components. The source Word report documents an R/Random Forest/Shiny implementation; the additional components shown in the image are not verified as implemented.

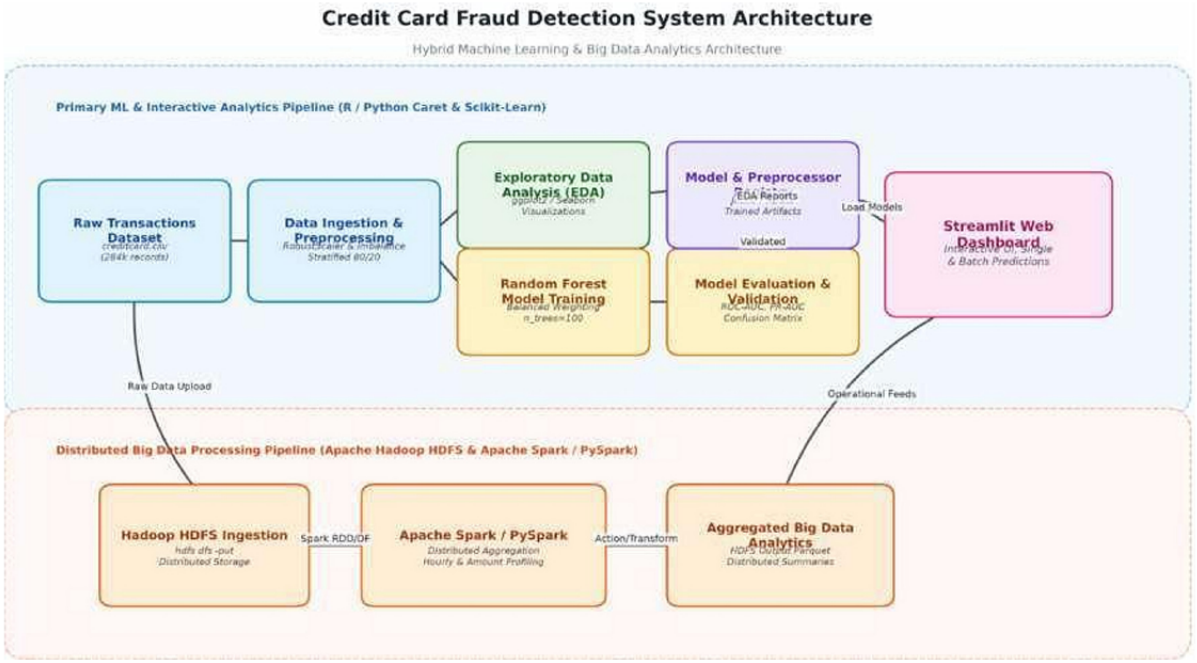



**Figure 1. Credit Card Fraud Detection System Architecture (supplied screenshot)**

**3.1.1 Data Source.** The Kaggle credit card transaction dataset stored as a CSV file.

**3.1.2 Data Loading and Validation.** The CSV file is loaded into R. The system checks that all required columns (Time, V1 to V28, Amount, Class) exist, that Class contains only 0 and 1, and that the predictors are numeric.

**3.1.3 Data Preprocessing.** Duplicate rows and rows with missing values are removed, the Class column is converted to a labelled factor (Legitimate and Fraud), and the data is split into training and testing sets using stratified sampling.

**3.1.4 Imbalance Handling.** The training set is balanced by undersampling the legitimate class. The test set is left untouched.

**3.1.5 Model Training.** A Random Forest model is tuned with caret cross-validation and trained with the randomForest package. The trained model is saved as an RDS file.

**3.1.6 Model Evaluation.** The saved model predicts on the untouched test set, and performance metrics and feature importance are calculated.

**3.1.7 Shiny Web Application.** The application loads the saved model and presents the dashboard, charts, performance results, feature importance, single-transaction prediction and batch prediction.

Overall flow: Credit Card Dataset (CSV) -> Data Loading and Validation -> Preprocessing -> Train/Test Split -> Imbalance Handling (training data only) -> Random Forest Training -> Saved Model -> Evaluation -> Shiny Application -> Prediction Results.

### 3.2 Data Flow Diagram - Level 0

The Level 0 DFD (context diagram) shows the whole system as one process, 0.0 Credit Card Fraud Detection System.

The source Word report refers to DFD and flowchart visuals on presentation slides 5-8. No matching DFD or flowchart image appears in the supplied screenshots PDF; the source descriptions below are therefore retained as text without reconstructed diagrams.

- External entities: Transaction Dataset (source of historical transaction data) and User (student, analyst or bank staff).
- Data store: D1 Trained Model and Results Store.
- Data flows: (1) Transaction Dataset to System: raw transaction data. (2) System to D1: trained model and evaluation results. (3) D1 to System: stored model for prediction. (4) User to System: prediction request (single transaction or CSV file). (5) System to User: predicted class, fraud probability, charts and performance results.

### 3.3 Data Flow Diagram - Level 1

The Level 1 DFD divides the system into its main processes:

- Process 1.0 Data Loading: reads the CSV file and checks that the file and columns exist. Raw data is stored in D1 Raw Data Store.
- Process 2.0 Data Preprocessing: removes duplicates and missing rows, converts the target column and splits the data. Output is stored in D2 Processed Data Store.
- Process 3.0 Model Training: balances the training data, tunes and trains the Random Forest and saves the model in D3 Model Store.
- Process 4.0 Model Evaluation: tests the model on the test set and stores metrics and feature importance in D4 Results Store.
- Process 5.0 Prediction and Visualization: accepts user input, validates it, uses the saved model to predict, and presents charts and results to the User.

Data stores: D1 Raw Data Store, D2 Processed Data Store, D3 Model Store, D4 Results Store.

### 3.4 Data Flow Diagram - Level 2

The Level 2 DFD expands the prediction process (Process 5.0):

**5.1 Receive Input: a sample transaction, manually entered values or an uploaded CSV file.**

**5.2 Validate Input: checks required columns, numeric values, missing values and negative Time or Amount values. Invalid rows are skipped and reported.**

**5.3 Arrange Features: places the valid values in the same order as used in training.**

**5.4 Predict: the saved Random Forest returns a fraud probability; a probability of 0.5 or higher is flagged as potentially fraudulent.**

**5.5 Display and Export Results: shows the predicted class, probability and explanation, counts flagged transactions and allows the results to be downloaded as a CSV file.**

### 3.5 System Flowchart

The system flowchart follows this sequence:

# Chapter IV: METHODOLOGY

The project follows a step-by-step process of loading, exploring, preparing, modelling, evaluating and presenting the data.

## 4.1 Dataset Collection

The Kaggle Credit Card Fraud Detection dataset (originally collected by Worldline and the Machine Learning Group of Universite Libre de Bruxelles) is used. It has 284,807 rows and 31 columns: Time (seconds since the first transaction), V1 to V28 (anonymized PCA components), Amount (transaction amount) and Class (target). Because of confidentiality, the original features such as merchant or location are not available.

Dataset inspection reports 0 missing values and 1,081 duplicate rows.
The dataset counts in this report describe the original source file.

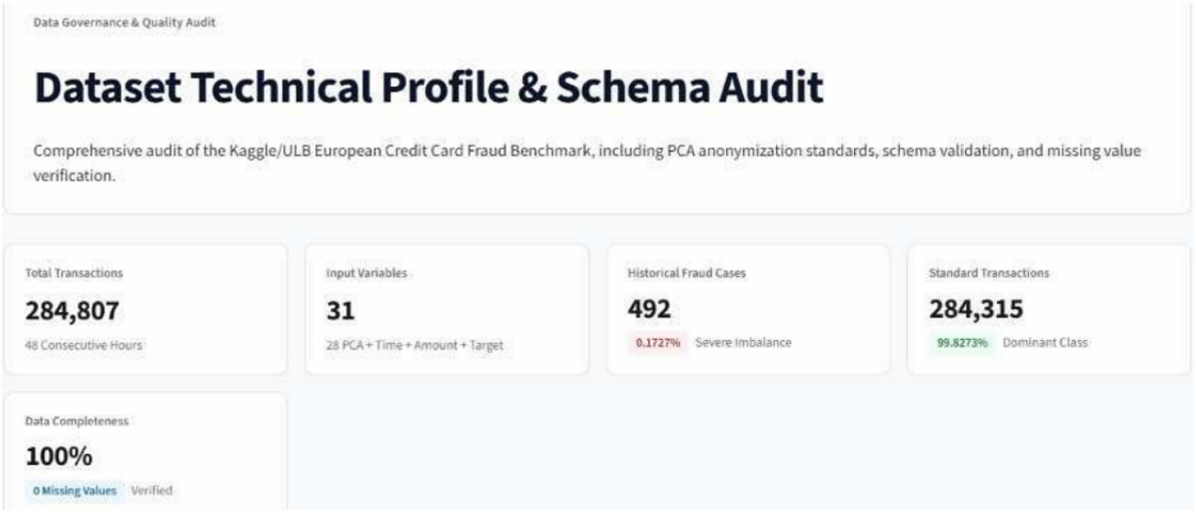



**Figure 2. Dataset Technical Profile and Schema Audit (website screenshot)**

## 4.2 Dataset Analysis and Visualization

Using dplyr, the project counts rows and columns, checks data types, missing values and duplicate rows, and counts legitimate and fraudulent transactions. The fraud percentage is calculated to show the class imbalance. Using ggplot2, the following charts are produced: legitimate versus fraudulent transactions, transaction amount distribution (log scale), amount comparison by class (boxplot), transaction time distribution by class, and the features most correlated with fraud.

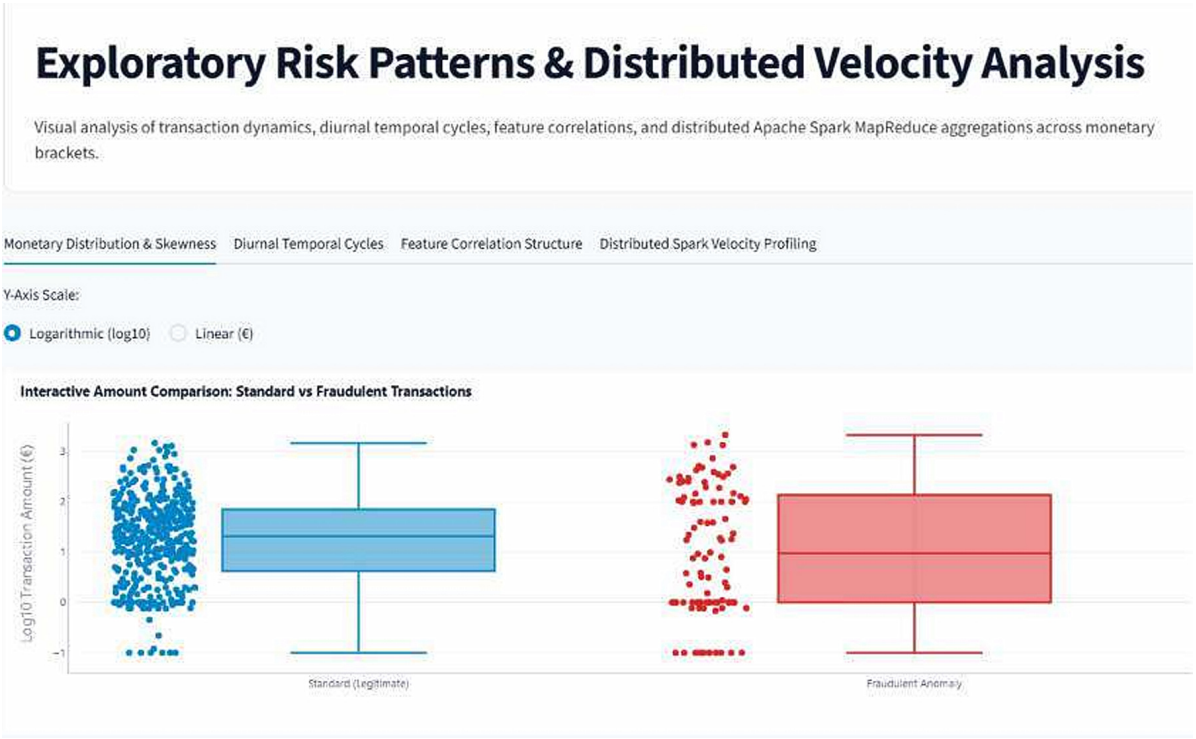



**Figure 3. Exploratory Risk Patterns: Transaction Amount Comparison (supplied screenshot)**

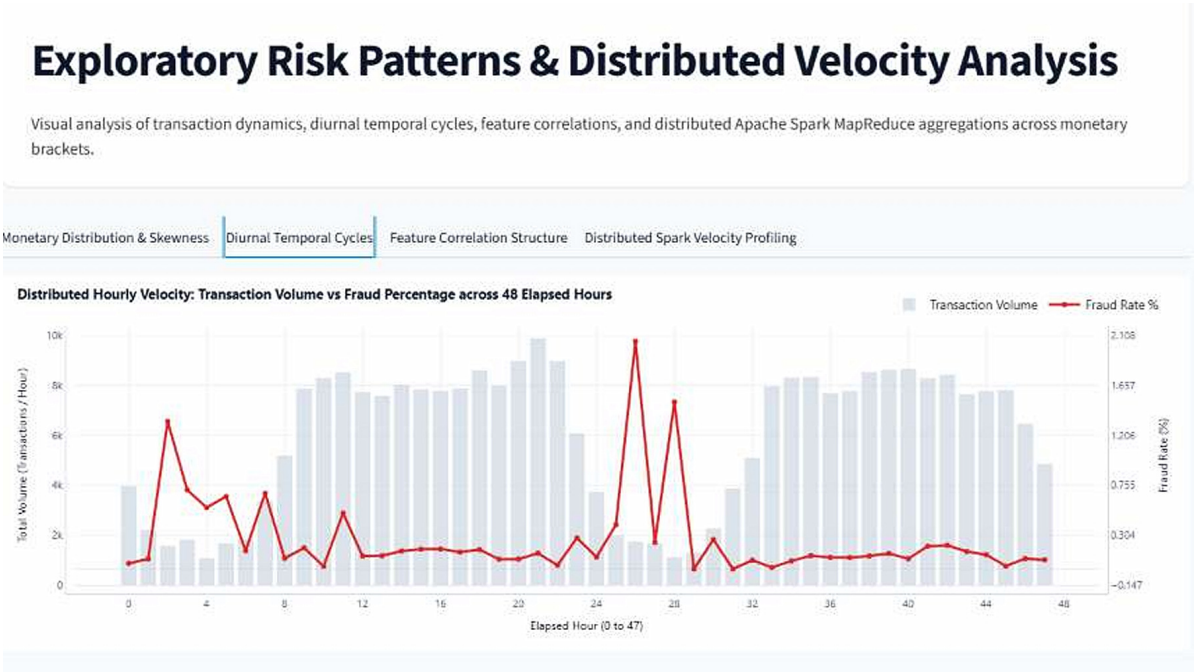



**Figure 4. Exploratory Risk Patterns: Temporal Transaction View (supplied screenshot)**

Note. The supplied website screenshot shows Spark-based aggregation and amount-bracket summaries. The screenshot is reproduced as provided. Confirm the Spark run and currency labels against the project implementation.

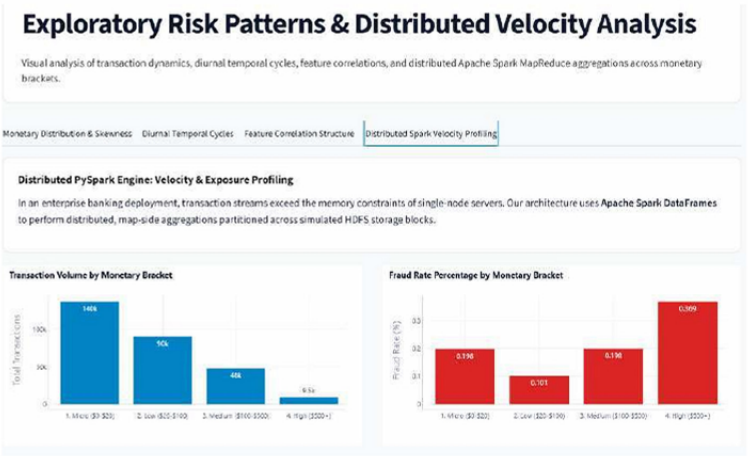



**Figure 5. Distributed Spark Analytics Summary (website screenshot)**

### 4.3 Data Preprocessing

- The dataset is validated for required columns and valid Class values.
- Class 0 is labelled Legitimate and Class 1 is labelled Fraud.
- Exact duplicate rows are removed, because the same transaction appearing in both training and testing sets would leak information. Rows with missing predictor values are removed because Random Forest cannot use them. No other data is deleted.
- No feature scaling is applied, because Random Forest is not based on distances.

### 4.4 Train-Test Split

The data is divided into 80% training and 20% testing data using stratified sampling (caret createDataPartition) with a fixed random seed (42), so that both sets keep the same fraud proportion and results are reproducible.

### 4.5 Handling Class Imbalance

Because fraud is extremely rare, a model could ignore it and still appear accurate. To address this, random undersampling of the legitimate class is applied to the training set only: all fraud rows are kept and up to ten legitimate rows per fraud row are retained. The test set is never resampled, so that evaluation reflects realistic conditions.

### 4.6 Random Forest Model

A Random Forest builds many decision trees, each on a random sample of the data and features, and combines their votes to make the final decision. It handles many features, captures non-linear patterns, needs little preprocessing and provides feature importance. The

parameter mtry (number of features tried at each split) is tuned among 5, 10 and 15 using 5-fold cross-validation in caret on the training data. The final model is trained with the randomForest package (300 trees) and saved as an RDS file (models/fraud_model.rds) together with its feature list and decision threshold, so it can be loaded later without retraining.

### 4.7 Model Evaluation

The saved model predicts on the untouched test set. The following measures are calculated from the actual predictions:

- Confusion matrix: true positives, false positives, true negatives and false negatives.
- Accuracy: share of all predictions that are correct. It can be misleading here, since predicting every transaction as legitimate would already give very high accuracy.
- Precision: of the transactions flagged as fraud, the share that really are fraud.
- Recall (sensitivity): of all real frauds, the share that are caught.
- Specificity: of all legitimate transactions, the share correctly left unflagged.
- F1-score: the balance between precision and recall.
- ROC-AUC and PR-AUC: how well the model ranks fraud above legitimate transactions across all thresholds; PR-AUC is especially informative for rare classes.

### 4.8 Feature Importance

Random Forest importance (Mean Decrease Gini and Mean Decrease Accuracy) shows which features the model used most. Since V1 to V28 are anonymized PCA components, their real-world meaning is unknown, and importance does not prove that a feature causes fraud.

### 4.9 Shiny Web Application

The Shiny application loads the saved model and results and contains: a Home dashboard with summary cards and project introduction; Dataset statistics; Charts; Model Performance; Feature Importance; Predict Transaction (select a held-out sample or edit all 30 input values, with validation); and Batch Prediction (upload a CSV, validate, predict, count flagged transactions and download results). Predictions are presented as statistical estimates and not as proof of fraud.

### 4.10 Overall Methodology

Credit Card Dataset -> Analysis and Visualization -> Preprocessing -> Stratified Split -> Imbalance Handling (training only) -> Random Forest Training -> Saved Model -> Evaluation on Test Set -> Shiny Application -> Prediction Results.

# Chapter V: SYSTEM REQUIREMENTS

## 5.1 Hardware Requirements

| Component | Requirement |
|---|---|
| Processor | Intel Core i5 or equivalent |
| RAM | Minimum 4 GB |
| Storage | Minimum 10 GB free space |
| Input/Output | Standard keyboard, mouse and display monitor |

## 5.2 Software Requirements

| Software | Purpose |
|---|---|
| Operating System | Windows / Linux / macOS |
| R | Programming language for data analysis and machine learning |
| RStudio | Development environment |
| dplyr | Data cleaning and manipulation |
| ggplot2 | Data visualization |
| caret | Machine learning workflow, cross-validation and evaluation |
| randomForest | Random Forest classification |
| e1071 | Supporting package used by caret |
| shiny | Interactive web application |
| Dataset | Kaggle Credit Card Fraud Detection (creditcard.csv) |

## 5.3 Project Structure

The project is organized into separate scripts: config.R (settings), data_preprocessing.R, explore_data.R, plots.R, train_model.R, utils_metrics.R, model_evaluation.R, prediction.R, app.R, a tests folder, a models folder for the saved model and an outputs folder for charts and results.

# Chapter VI: RESULTS AND OUTPUTS

Dataset profile values below describe the full source file before preprocessing.
The evaluation values are taken from the supplied project results and screenshots.
Confirm run-specific counts against the final R/Shiny implementation.

## 6.1 Dataset Overview

| Parameter | Result |
|---|---|
| Total transactions | 284,807 |
| Total columns | 31 |
| Missing values | 0 |
| Duplicate rows | 1,081 |
| Legitimate transactions | 284,315 |
| Fraudulent transactions | 492 |
| Fraud percentage | 0.1727% |

The very small fraud percentage confirms that the dataset is highly imbalanced.

## 6.2 Visualizations

The source report identifies the following saved chart outputs. Their image files were not included in the supplied attachments; the related dashboard screenshots included in this report are identified separately.

- Source-listed chart image not supplied: 01_class_balance.png - legitimate versus fraudulent transactions.
- Source-listed chart image not supplied: 02_amount_hist.png - transaction amount distribution.
- Source-listed chart image not supplied: 03_amount_boxplot.png - transaction amount by class.
- Source-listed chart image not supplied: 04_time_density.png - transaction time distribution by class.
- Source-listed chart image not supplied: 05_correlation.png - features most correlated with fraud.

Interpretations of the five saved chart files were not supplied because the files themselves were not included.

### 6.3 Model Evaluation Results

| Metric | Value |
|---|---|
| Accuracy | 99.937% |
| Precision | 81.00% |
| Recall | 82.65% (81 of 98) |
| Specificity | 99.967% |
| F1-score | 0.8182 |
| ROC-AUC | 0.9614 |
| PR-AUC | 0.8418 |

The supplied evaluation output reports 56,962 held-out transactions, including 98 fraud cases. Values are included as requested; confirm they match the final R model run described here. The figures below show evaluation curves, comparative performance, and feature importance from the supplied website screenshots.

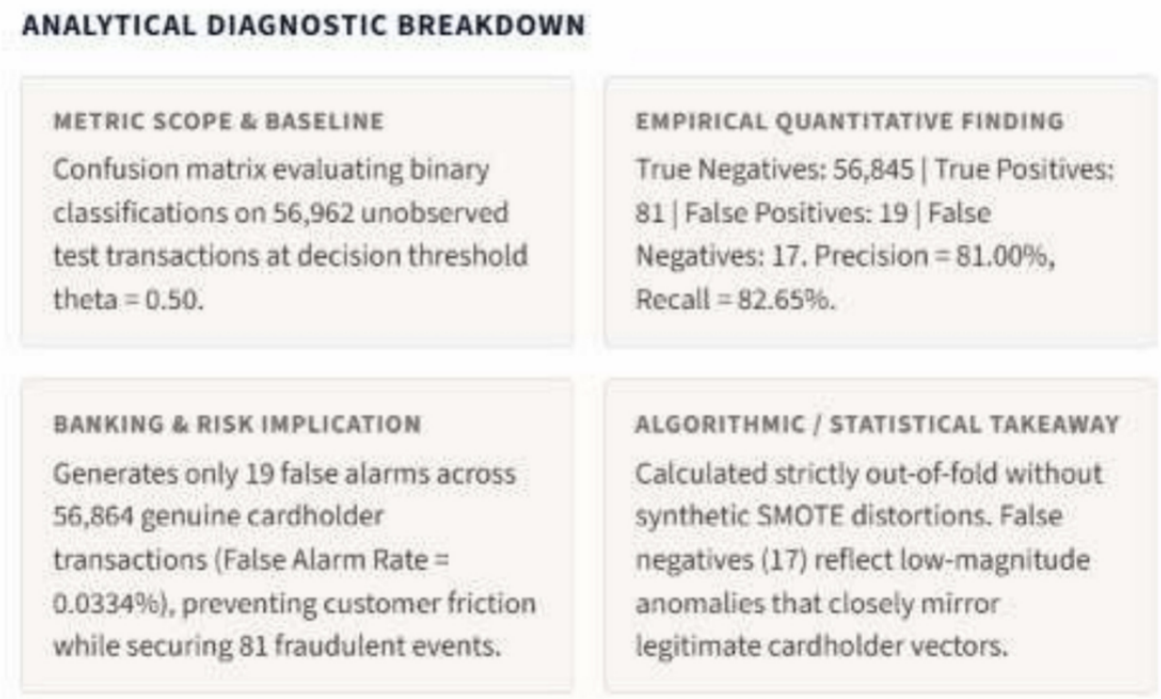



**Figure 6. Diagnostic Validation Summary (low-resolution supplied screenshot)**

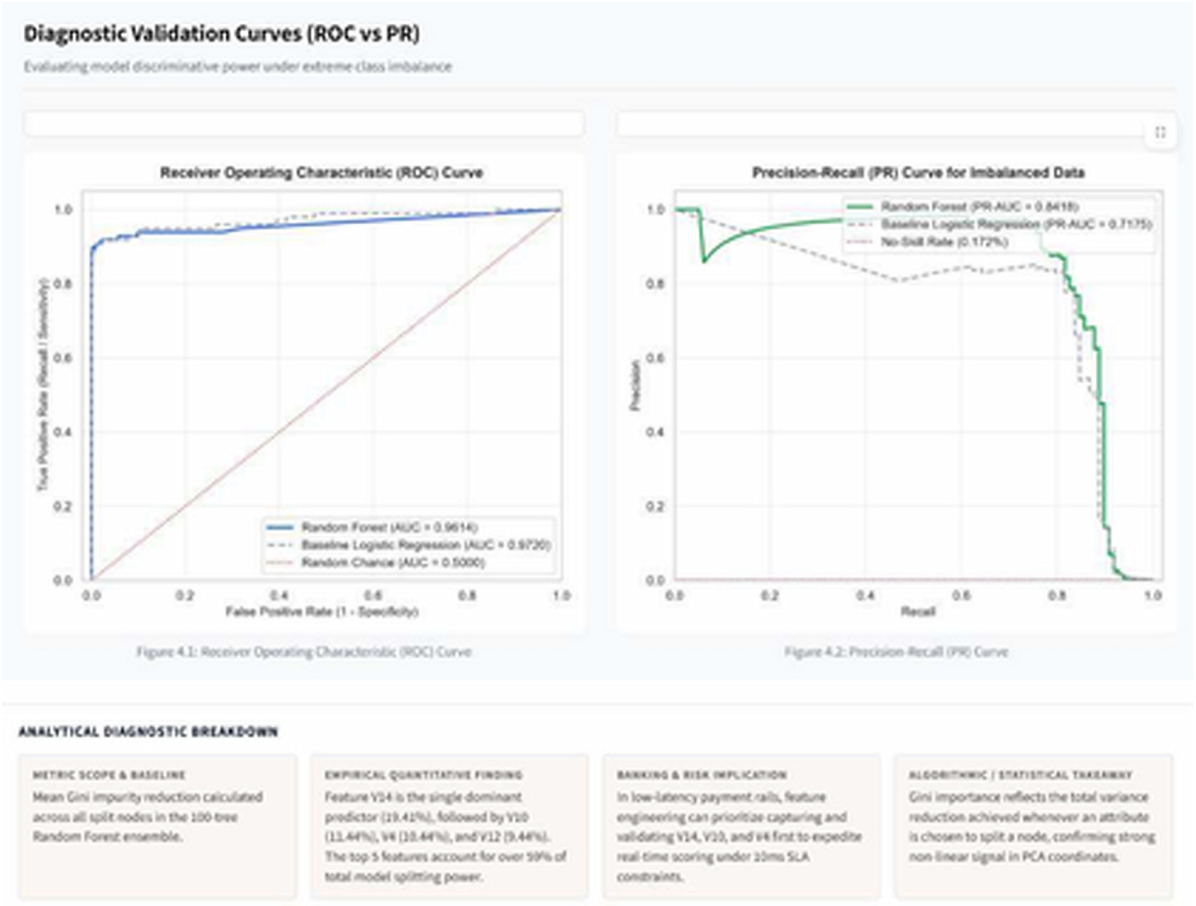



**Figure 7. Diagnostic Validation Curves: ROC and Precision-Recall (supplied screenshot)**

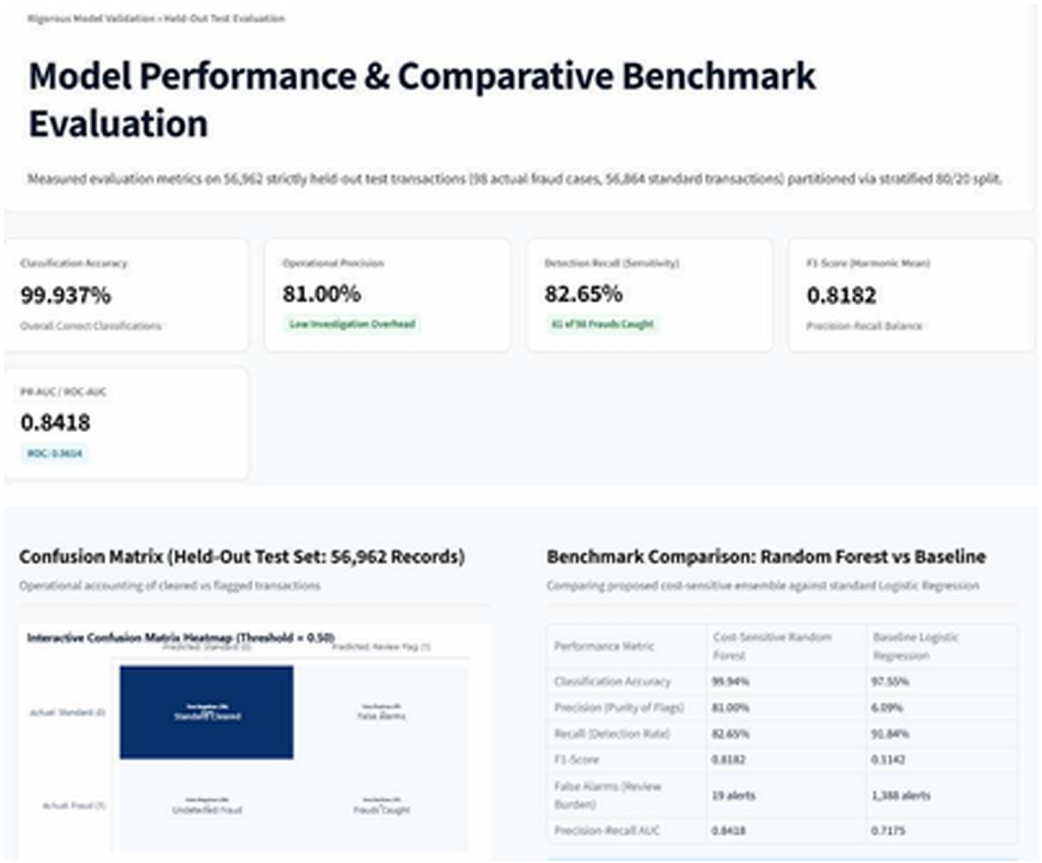



**Figure 8. Model Performance and Comparative Benchmark Evaluation (supplied screenshot)**

### 6.4 Feature Importance

The supplied feature-importance screenshot is reproduced below. Its feature ranking is shown as displayed and has not been independently verified against a saved model artifact.




**Figure 9. Ensemble Feature Importance (supplied screenshot)**

### 6.5 Shiny Application Screenshots

The supplied screenshots include transaction scoring and monitoring interfaces. They are reproduced as supplied; the screenshots show interface elements and technologies not documented in the source Word report, so they are not treated here as proof of a running Shiny application or live monitoring system.

Note. The screenshots below display a risk-scoring console and batch results. The source report describes a Shiny application; the screenshots' platform and output values have not been verified against that application.

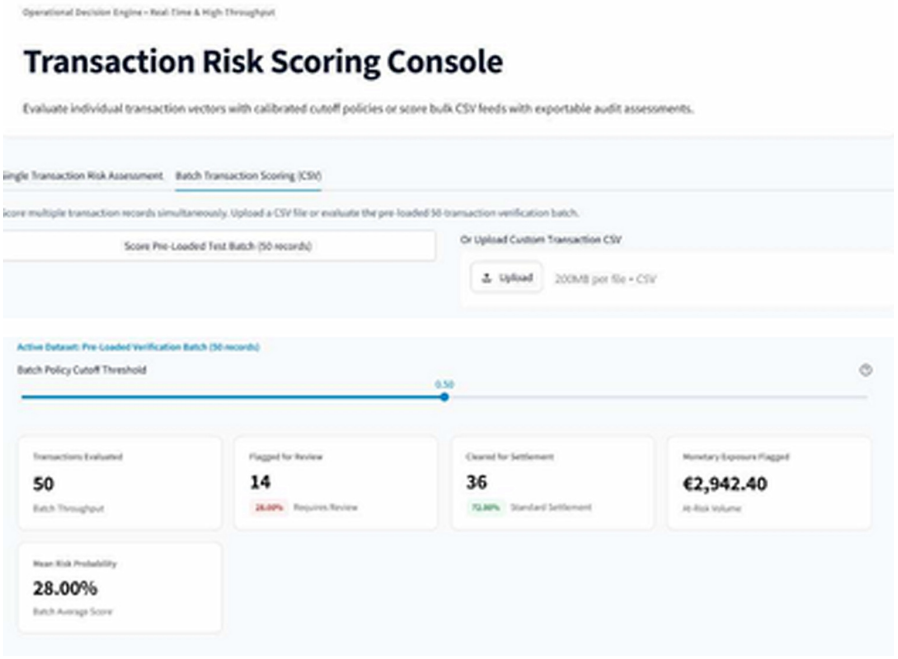
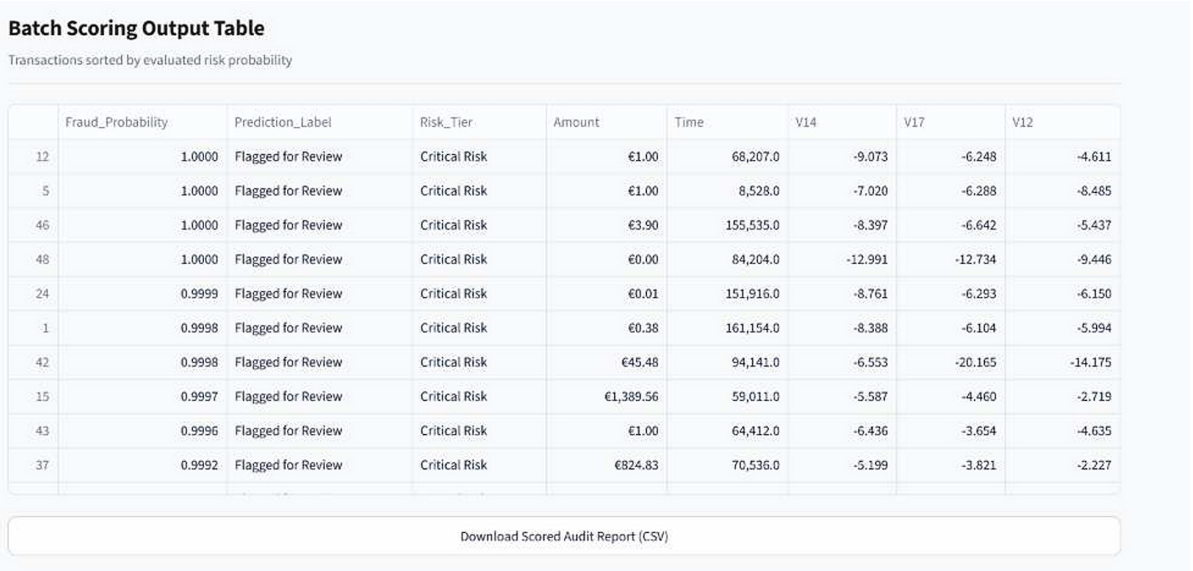



**Figure 10. Transaction Risk Scoring Console (supplied screenshot)**



**Batch Scoring Output Table**

Transactions sorted by evaluated risk probability

| | Fraud_Probability | Prediction_Label | Risk_Tier | Amount | Time | V14 | V17 | V12 |
|---|---|---|---|---|---|---|---|---|
| 12 | 1.0000 | Flagged for Review | Critical Risk | €1.00 | 68,207.0 | -9.073 | -6.248 | -4.611 |
| 5 | 1.0000 | Flagged for Review | Critical Risk | €1.00 | 8,528.0 | -7.020 | -6.288 | -8.485 |
| 46 | 1.0000 | Flagged for Review | Critical Risk | €3.90 | 155,535.0 | -8.397 | -6.642 | -5.437 |
| 48 | 1.0000 | Flagged for Review | Critical Risk | €0.00 | 84,204.0 | -12.991 | -12.734 | -9.446 |
| 24 | 0.9999 | Flagged for Review | Critical Risk | €0.01 | 151,916.0 | -8.761 | -6.293 | -6.150 |
| 1 | 0.9998 | Flagged for Review | Critical Risk | €0.38 | 161,154.0 | -8.388 | -6.104 | -5.994 |
| 42 | 0.9998 | Flagged for Review | Critical Risk | €45.48 | 94,141.0 | -6.553 | -20.165 | -14.175 |
| 15 | 0.9997 | Flagged for Review | Critical Risk | €1,389.56 | 59,011.0 | -5.587 | -4.460 | -2.719 |
| 43 | 0.9996 | Flagged for Review | Critical Risk | €1.00 | 64,412.0 | -6.436 | -3.654 | -4.635 |
| 37 | 0.9992 | Flagged for Review | Critical Risk | €824.83 | 70,536.0 | -5.199 | -3.821 | -2.227 |

Download Scored Audit Report (CSV)

**Figure 11. Batch Risk Scoring Output Table (supplied screenshot)**

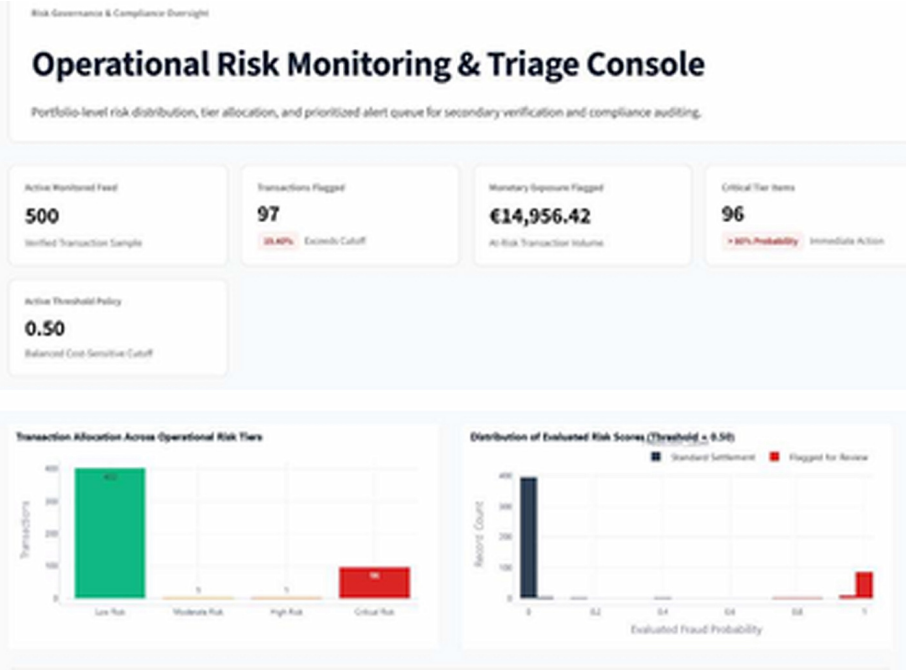



**Figure 12. Operational Risk Monitoring and Triage Console (supplied screenshot)**

**Prioritized Compliance Review Queue (Ranked by Risk Score)**

Transactions flagged for human compliance verification or secondary step-up authorization

| | Fraud_Probability | Risk_Tier | Amount | Time | V14 | V17 | V12 | V10 |
|---|---|---|---|---|---|---|---|---|
| 13 | 1.0000 | Critical Risk | €0.00 | 84,204.0 | -12.991 | -12.734 | -9.446 | -8.099 |
| 77 | 1.0000 | Critical Risk | €252.92 | 85,285.0 | -11.608 | -11.939 | -8.682 | -8.538 |
| 34 | 1.0000 | Critical Risk | €0.00 | 85,285.0 | -11.534 | -11.888 | -8.577 | -8.410 |
| 39 | 1.0000 | Critical Risk | €3.90 | 155,535.0 | -8.397 | -6.642 | -5.437 | -3.927 |
| 44 | 1.0000 | Critical Risk | €188.52 | 93,860.0 | -14.224 | -15.842 | -16.466 | -14.925 |
| 47 | 1.0000 | Critical Risk | €252.92 | 85,285.0 | -11.608 | -11.939 | -8.682 | -8.538 |
| 50 | 1.0000 | Critical Risk | €9.82 | 97,235.0 | -3.547 | -18.372 | -12.894 | -15.232 |
| 46 | 1.0000 | Critical Risk | €99.99 | 28,692.0 | -5.080 | -11.275 | -5.661 | -7.870 |
| 59 | 1.0000 | Critical Risk | €1.00 | 94,362.0 | -3.926 | -12.462 | -10.592 | -22.187 |
| 67 | 1.0000 | Critical Risk | €1.00 | 8,528.0 | -7.020 | -6.288 | -8.485 | -5.390 |

**Figure 13. Prioritized Compliance Review Queue (supplied screenshot)**

# Chapter VII: CONCLUSION

The Credit Card Fraud Detection project was developed to show how machine learning can be applied to identify potentially fraudulent credit card transactions. The Kaggle credit card dataset was analyzed, visualized and preprocessed, and a Random Forest classifier was trained and evaluated using R, dplyr, ggplot2 and caret.

The project handled the severe class imbalance by resampling only the training data, prevented data leakage through a stratified split and removal of duplicate rows, and evaluated the model on an untouched test set using precision, recall, F1-score, ROC-AUC and PR-AUC, because accuracy alone is misleading when fraud is rare. The trained model was saved and reused in an interactive Shiny application that supports single-transaction and batch CSV prediction with input validation.

**Limitations.** The dataset is historical, anonymized and limited to two days of European card transactions, so performance may not carry over to other banks, regions or periods. Merchant category, customer spending history and location are not available. A fixed probability threshold of 0.5 is used, whereas a bank would choose it based on the cost of missed fraud versus false alarms. The application is a demonstration and is not connected to any live banking system; its output is a statistical estimate and not proof of fraud.

**Future scope.** The project can be extended by comparing other algorithms (logistic regression, XGBoost), trying SMOTE or class weights, tuning the decision threshold, using time-based validation, adding anomaly detection, and building a real-time scoring service.

# REFERENCES


- Machine Learning Group - ULB and Worldline, Credit Card Fraud Detection dataset, Kaggle. Primary source of transaction data.
- R Core Team, R: A Language and Environment for Statistical Computing, R Foundation for Statistical Computing.
- Posit, RStudio IDE Documentation. Development environment.
- A. Liaw and M. Wiener, randomForest: Breiman and Cutler's Random Forests for Classification and Regression, R package documentation.
- M. Kuhn, caret: Classification and Regression Training, R package documentation.
- H. Wickham et al., dplyr: A Grammar of Data Manipulation, R package documentation.
- H. Wickham, ggplot2: Elegant Graphics for Data Analysis, R package documentation.
- Posit, Shiny: Web Application Framework for R, documentation.
- L. Breiman, Random Forests, Machine Learning, vol. 45, 2001.
- Project source code and outputs developed by the project team as part of the Credit Card Fraud Detection project.